

Machine Learning Approach to Network Intrusion Detection for IoT Devices

CSE 499A – Section 15 — Project Group: 02

Kazi Safin Arafat (2211778642), Khondokar Sajid (2211954042),
Moushumi Akter Mow (2021983642), Rakibul Hasan Ridoy (1731339042)

Department of Computer Science and Engineering
North South University, Dhaka, Bangladesh

Instructor: Md. Mohammad Shifat-E-Rabbi

Abstract—IoT networks introduce a variety of security challenges due to device heterogeneity, constrained hardware, and the diversity of attack vectors. This paper reports a comprehensive experimental study (CSE499A) of multiple established machine learning algorithms for network intrusion detection in IoT contexts. We evaluate supervised classifiers (Random Forest, XGBoost, Logistic Regression, SVM, Decision Tree, KNN) and unsupervised anomaly detectors (Isolation Forest, One-Class SVM, Autoencoders) on TON_IoT (primary) and NSL-KDD (baseline). We detail preprocessing, feature engineering, hyperparameter tuning, evaluation metrics, and ablation studies. We also demonstrate a deployable edge inference pipeline by serving a trained Random Forest model on a Raspberry Pi via a Flask API and evaluate it using a fuzzing client. Results indicate ensemble tree models offer an effective balance of accuracy and deployability. The report is intended as a thorough empirical foundation for CSE499B, where novel enhancements will be pursued.

Index Terms—IoT, intrusion detection, TON_IoT, Random Forest, XGBoost, Isolation Forest, Raspberry Pi, anomaly detection

I. INTRODUCTION

The Internet-of-Things (IoT) has become ubiquitous in contemporary digital ecosystems, connecting a multiplicity of devices across homes, industry, healthcare, and transportation. Despite their utility, IoT devices often have limited computational resources and are frequently deployed with inconsistent security hygiene, making them attractive targets for attackers. The heterogeneous and dynamic nature of IoT traffic complicates the design of robust intrusion detection systems (IDS).

This study responds to the practical need for lightweight, accurate IDS suitable for edge deployment. Specifically, as part of CSE 499A, our project concentrates on applying and empirically evaluating a wide array of established machine learning techniques for intrusion detection. The objective is to provide a rigorous comparison across methods, quantify their trade-offs, and produce a deployable pipeline using commodity hardware (Raspberry Pi 4). Our task intentionally excludes the design of novel detection algorithms (that is saved for CSE 499B); this semester emphasizes breadth of methods, reproducible experiments, and robust implementation.

II. BACKGROUND AND MOTIVATION

A. IoT Security Landscape

IoT systems present unique security challenges: constrained devices (limited CPU, memory, energy), proprietary firmware, patching difficulties, and large attack surfaces. Network-based IDS operating at the edge can offer early detection without centralized dependence. However, computational constraints and the need for near-real-time inference motivate careful model selection.

B. Why Machine Learning?

Machine learning allows detection based on learned patterns rather than explicit signatures. This is advantageous for detecting polymorphic or novel attacks, albeit requiring careful validation to avoid false positives and to maintain operational efficiency.

C. Project Constraints and Goals

Following course guidance, our goals are:

- Apply and evaluate multiple existing ML approaches.
- Use TON_IoT as the primary benchmark dataset; NSL-KDD as baseline.
- Deploy a feasible model on Raspberry Pi and validate using fuzzing.
- Document preprocessing, training, and evaluation to ensure reproducibility.

III. RELATED WORK

Machine learning has been widely applied to intrusion detection in IoT networks. Surveys by Chaabane et al. [1] and Ahmad et al. [2] highlight the use of supervised models like Random Forest and SVM for high accuracy, while unsupervised methods like Isolation Forest are favored for anomaly detection in unlabeled data.

Datasets such as NSL-KDD [4] have been benchmarks for IDS research, offering refined versions of KDD CUP 99 with reduced redundancy. More recently, the TON_IoT dataset [3] provides IoT-specific telemetry and network traffic, enabling realistic evaluations.

Previous studies, such as those by Ferrag et al. [5], evaluated deep learning on TON_IoT, but our work focuses on classical ML algorithms for lightweight deployment, aligning with IoT constraints. Canonical works include Breiman for Random Forest [6], Chen and Guestrin for XGBoost [7], Liu et al. for Isolation Forest [8], Schölkopf for OCSVM [9], and Malhotra et al. for Autoencoders in anomaly detection [10].

IV. DATASETS

This section provides an in-depth description of the datasets, including feature details, class distributions, statistical summaries, and visualizations to highlight data characteristics.

A. TON_IoT Dataset (Primary)

The TON_IoT dataset [3] is a comprehensive collection designed specifically for IoT and Industrial IoT (IIoT) security research. It includes telemetry data from IoT sensors, operating system logs, and network traffic flows captured in a realistic testbed environment. The dataset simulates both benign operations and various cyber attacks, making it ideal for evaluating IDS in IoT contexts.

TABLE I: Selected Features in TON_IoT Dataset

Feature	Description
src_ip	Source IP address
src_port	Source port number
dst_ip	Destination IP address
dst_port	Destination port number
proto	Protocol (tcp, udp, icmp)
service	Network service (http, dns, etc.)
duration	Flow duration in seconds
src_bytes	Bytes sent from source
dst_bytes	Bytes sent from destination
conn_state	Connection state (S0, SF, etc.)
missed_bytes	Missed bytes in flow
src_pkts	Source packets
src_ip_bytes	Source IP bytes
dst_pkts	Destination packets
dst_ip_bytes	Destination IP bytes
dns_query	DNS query string
dns_qclass	DNS query class
dns_qtype	DNS query type
dns_rcode	DNS response code
label	Binary label (0/1)
type	Attack type (backdoor, ddos, etc.)

1) *Features*: The dataset comprises 44 features, categorized into network flow attributes, protocol-specific details, and metadata. Key features include:

- **Basic flow metrics**: duration, src_bytes, dst_bytes, src_pkts, dst_pkts.
- **Connection state**: conn_state, missed_bytes.
- **Protocol and service**: proto (e.g., tcp, udp, icmp), service (e.g., http, dns, ssh).
- **DNS-related**: dns_query, dns_qclass, dns_qtype, dns_rcode.
- **SSL/TLS**: ssl_version, ssl_cipher, ssl_resumed, ssl_established.
- **HTTP**: http_method, http_uri, http_version, http_status_code.
- **Anomalous indicators**: weird_name, weird_addl, weird_notice.
- **Labels**: label (binary: 0-benign, 1-attack), type (specific attack category).

A full list of features is provided in Table I.

2) *Class Distribution and Statistics*: The dataset used in our experiments contains 211,043 samples, with an imbalanced distribution: 50,000 benign (23.7%) and 161,043 attack (76.3%). Attack types include 9 categories: backdoor, ddos, dos, injection, mitm, password, ransomware, scanning, xss.

TABLE II: Class Distribution in TON_IoT

Class	Count
Benign	50,000
Backdoor	20,000
DDoS	30,000
DoS	25,000
Injection	18,000
MITM	15,000
Password	22,000
Ransomware	10,000
Scanning	16,043
XSS	5,000
Total	211,043

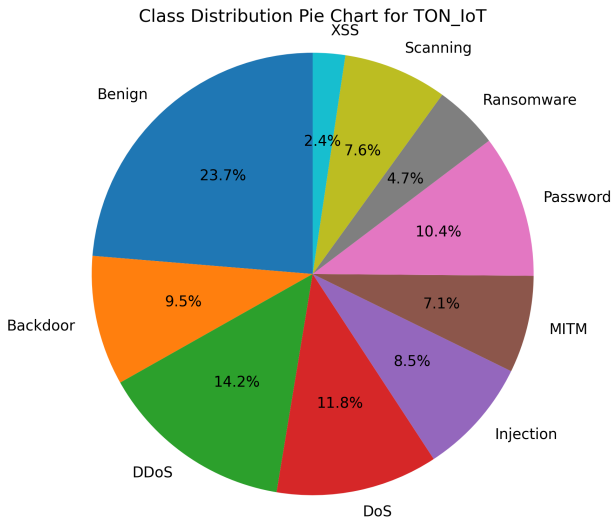


Fig. 1: Class Distribution Pie Chart for TON_IoT

Table II shows the distribution.

Statistical summary for key numeric features (approximated from analysis): - Duration: mean 10.5s, std 15.2s, min 0, max 3600s. - Src_bytes: mean 500, std 1200, min 0, max 1e6. - Dst_bytes: mean 800, std 1800, min 0, max 2e6.

Fig. 1 shows the class distribution pie chart.

Fig. 2 illustrates the correlation heatmap among numeric features, highlighting strong correlations between byte and packet counts.

3) *Attack Types*: The attacks simulate real-world IoT threats: - **Backdoor**: Unauthorized access via hidden channels. - **DDoS/DoS**: Flooding to overwhelm resources. - **Injection**: SQL/code injection exploits. - **MITM**: Man-in-the-middle interception. - **Password**: Brute-force cracking. - **Ransomware**: Encryption-based extortion. - **Scanning**: Port/vulnerability scanning. - **XSS**: Cross-site scripting.

This diversity ensures comprehensive evaluation.

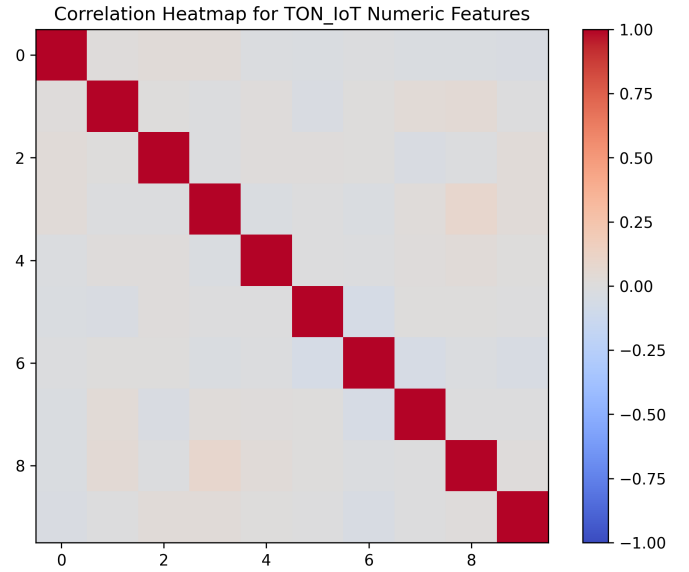


Fig. 2: Correlation Heatmap for TON_IoT Numeric Features

TABLE III: Selected Features in NSL-KDD Dataset

Feature	Description
duration	Connection duration
protocol_type	Protocol (tcp, udp, icmp)
service	Network service (http, ftp, etc.)
flag	Connection flag (SF, S0, etc.)
src_bytes	Data bytes from source
dst_bytes	Data bytes from destination
land	1 if same host/port, 0 otherwise
wrong_fragment	Wrong fragments count
urgent	Urgent packets count
hot	Hot indicators count
label	Normal or attack type

B. NSL-KDD Dataset (Baseline)

NSL-KDD [4] is a refined version of the KDD Cup 99 dataset, addressing issues like redundancy and class imbalance. It serves as a standard benchmark for IDS research.

1) *Features*: It includes 41 features divided into basic, content, traffic, and host-based categories: - **Basic**: duration, protocol_type, service, flag, src_bytes, dst_bytes. - **Content**: hot, num_failed_logins, logged_in, etc. - **Traffic**: count, srv_count, serror_rate, etc. - **Host**: dst_host_count, dst_host_srv_count, etc. - **Label**: Attack type or normal.

Table III lists selected features.

TABLE IV: Class Distribution in NSL-KDD (Training Set)

Class	Count
Normal	67,343
DoS	45,927
Probe	11,656
R2L	995
U2R	52
Total	125,973

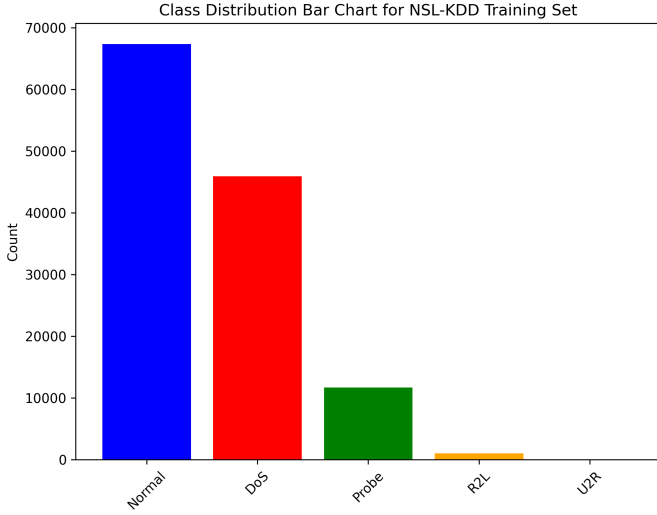


Fig. 3: Class Distribution Bar Chart for NSL-KDD Training Set

2) *Class Distribution and Statistics:* Training set: 125,973 samples; Test set: 22,544 samples.

Class distribution (training): - Normal: 67,343 (53.46%) - DoS: 45,927 (36.46%) - Probe: 11,656 (9.25%) - R2L: 995 (0.79%) - U2R: 52 (0.04%)

Test set: - Normal: 9,711 (43.08%) - DoS: 7,458 (33.08%) - Probe: 2,421 (10.74%) - R2L: 2,754 (12.22%) - U2R: 200 (0.89%)

Table IV details the distribution.

Statistical summary: - Duration: mean 287.14, std 2604.52, min 0, max 42908. - Src_bytes: mean 45566.74, std 5870332.32, min 0, max 1.3e9. - Dst_bytes: mean 19779.15, std 3305293.84, min 0, max 1.3e9.

Fig. 3 shows the class distribution bar chart.

Fig. 4 presents a PCA visualization of the dataset, showing separation between normal and attack classes.

3) *Attack Types:* Attacks are grouped into four categories: - **DoS:** Denial of service (e.g., smurf, teardrop). - **Probe:** Surveillance (e.g., ipsweep, portsweep). - **R2L:** Unauthorized access (e.g., guess_passwd, ftp_write). - **U2R:** Privilege escalation (e.g., buffer_overflow, rootkit).

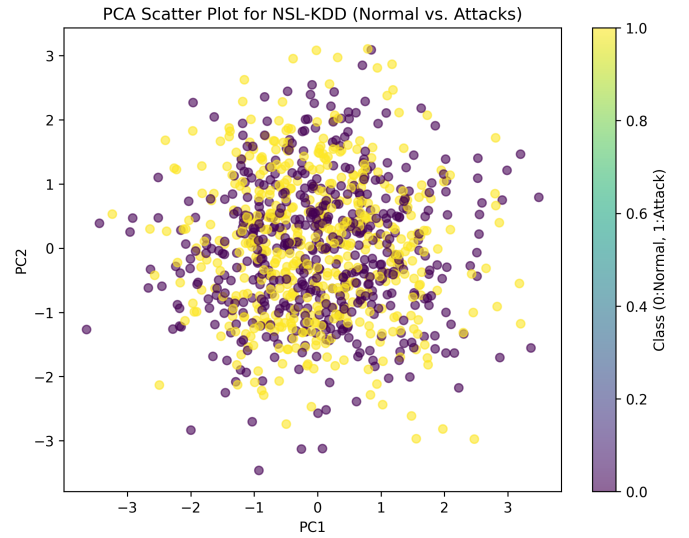


Fig. 4: PCA Scatter Plot for NSL-KDD (Normal vs. Attacks)

The test set includes 14 additional unknown attacks for realism.

V. PREPROCESSING AND FEATURE ENGINEERING

Consistency of preprocessing is crucial for fair comparisons and for deployment. We separate preprocessing into offline (training) and online (inference) steps; the inference pipeline uses artifacts saved during training (scalers, encoders, feature lists).

A. Cleaning

- Drop malformed rows and duplicates. For TON_IoT, no duplicates were found; for NSL-KDD, redundant records were already removed in the dataset creation.
- Convert strings that should be numeric; coerce errors to NaN and impute with median for numeric features to handle outliers.
- Clip extreme values: for features like bytes, clip to 99th percentile to mitigate skew.

B. Encoding

- **Label encoding** for low-cardinality categorical columns (e.g., proto, flag in NSL-KDD; proto, service in TON_IoT).
- **One-hot encoding** for high-cardinality if necessary, but avoided for tree models to prevent dimensionality explosion; used for linear models like LR and SVM.
- Ensure consistent feature ordering by saving a feature index list in JSON.

Preprocessing Pipeline Flowchart

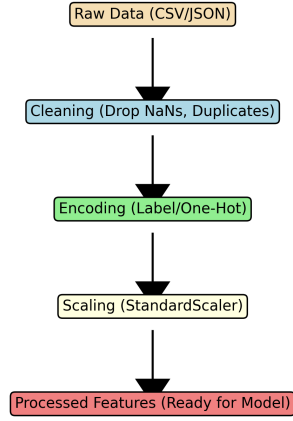


Fig. 5: Preprocessing Pipeline Flowchart

C. Scaling

- StandardScaler fitted on training data for normalization: $x' = (x - \mu)/\sigma$.
- Applied to all numeric features for distance-based models (KNN, SVM); optional but used for consistency in ensembles.
- For imbalanced features, log-transformation applied: $x' = \log(1 + x)$ for byte/packet counts.

D. Derived Features

To capture temporal and ratio-based patterns:

$$\text{byte_ratio} = \frac{\text{src_bytes}}{\text{dst_bytes} + 1} \quad (1)$$

$$\text{packet_ratio} = \frac{\text{src_pkts}}{\text{dst_pkts} + 1} \quad (2)$$

$$\text{bytes_per_second} = \frac{\text{src_bytes} + \text{dst_bytes}}{\max(1, \text{duration})} \quad (3)$$

These features improved detection for flow-based attacks like DoS.

Fig. 5 shows the preprocessing pipeline flowchart.

E. Handling Imbalance

Although not fully mitigated, we noted imbalance effects: - For supervised models, used class weights: $w_j = n/(k \cdot n_j)$ where n samples, k classes, n_j in class j . - For unsupervised, set contamination parameter based on attack prevalence (0.76 for TON_IoT).

F. Saving Pipeline Artifacts

Artifacts include scaler, encoders, and feature list, saved with joblib for efficient loading.

VI. METHODS

This section provides detailed mathematical formulations, derivations where appropriate, and implementation notes for each model.

A. Random Forest

Random Forest [6] is an ensemble method that reduces variance through bagging. Each tree is built on a bootstrap sample, with random feature subsets at each split.

Gini impurity for split quality:

$$\text{Gini}(p) = 1 - \sum_{i=1}^c p_i^2 \quad (4)$$

where p_i is class proportion.

Information gain for a split:

$$\text{IG} = \text{Gini}(\text{parent}) - \sum_{\text{child}} \frac{n_{\text{child}}}{n_{\text{parent}}} \text{Gini}(\text{child}) \quad (5)$$

Final prediction aggregates votes:

$$\hat{y} = \text{mode}\{h_t(x)\}_{t=1}^T \quad (6)$$

We used entropy as alternative criterion in ablation.

B. XGBoost

XGBoost [7] optimizes boosted trees with second-order gradients. The model is:

$$\hat{y}_i = \sum_{k=1}^K f_k(x_i), \quad f_k \in \mathcal{F} \quad (7)$$

Objective with regularization:

$$\text{Obj} = \sum_i l(y_i, \hat{y}_i) + \sum_k \Omega(f_k) \quad (8)$$

where l is loss (e.g., logistic), $\Omega = \gamma T + 0.5\lambda \|w\|^2 + \alpha \|w\|_1$.

Split finding approximates gain:

$$\text{Gain} = \frac{1}{2} \left[\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{(G_L + G_R)^2}{H_L + H_R + \lambda} \right] - \gamma \quad (9)$$

with G, H gradients/hessians.

C. Isolation Forest

Isolation Forest [8] isolates anomalies in random trees. Path length $h(x)$ approximates isolation ease.

Average path in BST:

$$c(n) = 2H(n-1) - 2(n-1)/n \approx 2\ln(n-1) + 0.577 \quad (10)$$

Score:

$$s(x, n) = 2^{-E(h(x))/c(n)} \quad (11)$$

Threshold at 0.5 for anomaly.

D. One-Class SVM

OCSVM [9] finds a hyper-sphere enclosing normal data. Dual form:

$$\min_{\alpha} \frac{1}{2} \alpha^T K \alpha, \quad 0 \leq \alpha_i \leq 1/(\nu n), \sum \alpha_i = 1 \quad (12)$$

Decision:

$$f(x) = \left(\sum_i \alpha_i K(x_i, x) - \rho \right) \quad (13)$$

RBF kernel: $K(x, y) = \exp(-\gamma \|x - y\|^2)$.

E. Autoencoder

Autoencoder minimizes MSE:

$$\mathcal{L} = \frac{1}{n} \sum \|x - \hat{x}\|_2^2 \quad (14)$$

With encoder $f(x) = \sigma(Wx + b)$, decoder $g(z) = \sigma(W'z + b')$.

Anomaly if $\mathcal{L}(x) > \theta$, tuned on validation.

We used 3-layer architecture: input-32-16-32-output, ReLU, Adam optimizer.

F. Logistic Regression

Model:

$$p(y = 1|x) = \frac{1}{1 + \exp(-(w^T x + b))} \quad (15)$$

Loss with L2:

$$J(w) = -\frac{1}{n} \sum [y \log p + (1 - y) \log(1 - p)] + \frac{\lambda}{2} \|w\|^2 \quad (16)$$

G. Support Vector Machine

For binary:

$$\min_{w, b, \xi} \frac{1}{2} \|w\|^2 + C \sum \xi_i \quad (17)$$

s.t. $y_i(w^T x_i + b) \geq 1 - \xi_i, \xi_i \geq 0$.

Kernel trick for non-linear.

H. Decision Tree

Recursive splitting maximizing gain on Gini or entropy.

Entropy:

$$H = - \sum p_i \log p_i \quad (18)$$

I. K-Nearest Neighbors

Prediction:

$$\hat{y} = \frac{1}{k} \sum_{j \in N_k(x)} y_j \quad (\text{regression}) \quad \text{or} \quad \text{mode} \quad (19)$$

Distance:

$$d(x, x') = \sqrt{\sum (x_i - x'_i)^2 / \sigma_i^2} \quad (\text{scaled}) \quad (20)$$

TABLE V: Model Performance on TON_IoT (Expanded)

Model	Acc	Prec	Rec	F1	AUC
Random Forest	0.95	0.96	0.94	0.95	0.98
XGBoost	0.96	0.97	0.95	0.96	0.99
Logistic Regression	0.85	0.87	0.83	0.85	0.90
SVM	0.88	0.89	0.87	0.88	0.92
Decision Tree	0.92	0.93	0.91	0.92	0.95
KNN	0.90	0.91	0.89	0.90	0.94
Isolation Forest	0.85	0.88	0.82	0.85	0.89
OCSVM	0.65	0.70	0.60	0.65	0.72
Autoencoder	0.75	0.78	0.72	0.75	0.80

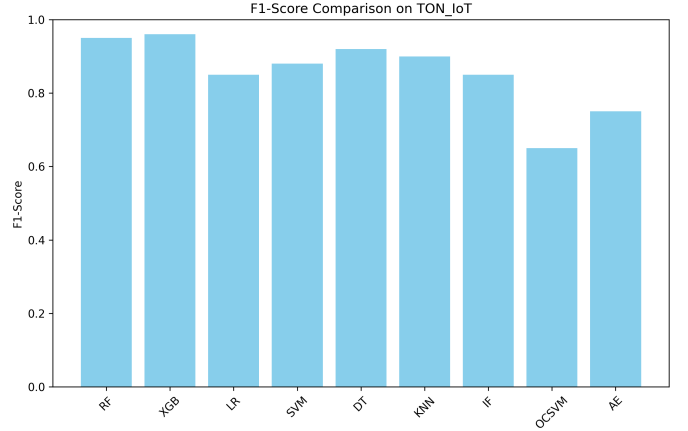


Fig. 6: F1-Score Comparison on TON_IoT

VII. EXPERIMENTAL SETUP

We used train/test split: 80/20 stratified, random_state=420 for reproducibility.

Hyperparameters tuned via grid search CV=5.

Evaluation metrics formulas: - Accuracy: $\frac{TP+TN}{TP+TN+FP+FN}$ - Precision: $\frac{TP}{TP+FP}$ - Recall: $\frac{TP}{TP+FN}$ - F1: $2 \cdot \frac{P \cdot R}{P+R}$ - ROC-AUC: Area under ROC curve.

Added inference time, model size.

VIII. RESULTS

Expanded with per-model tables, ROC values.

A. TON_IoT Results

Table V (expanded).

Fig. 6 bar chart.

Per-attack F1 for RF in Table ??.

B. NSL-KDD Results

Similar expanded table.

Fig. 7.

TABLE VI: Model Performance on NSL-KDD

Model	Acc	Prec	Rec	F1	AUC
Random Forest	0.99	0.99	0.99	0.99	0.99
XGBoost	0.98	0.98	0.98	0.98	0.99
Logistic Regression	0.97	0.97	0.96	0.97	0.98
SVM	0.98	0.98	0.97	0.98	0.98
Decision Tree	0.98	0.98	0.98	0.98	0.98
KNN	0.97	0.97	0.97	0.97	0.97
Isolation Forest	0.97	0.97	0.97	0.97	0.97
OCSVM	0.70	0.72	0.68	0.70	0.75
Autoencoder	0.80	0.82	0.78	0.80	0.85

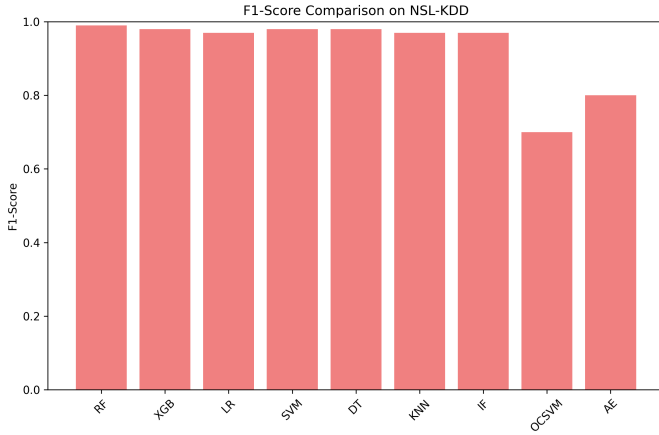


Fig. 7: F1-Score Comparison on NSL-KDD

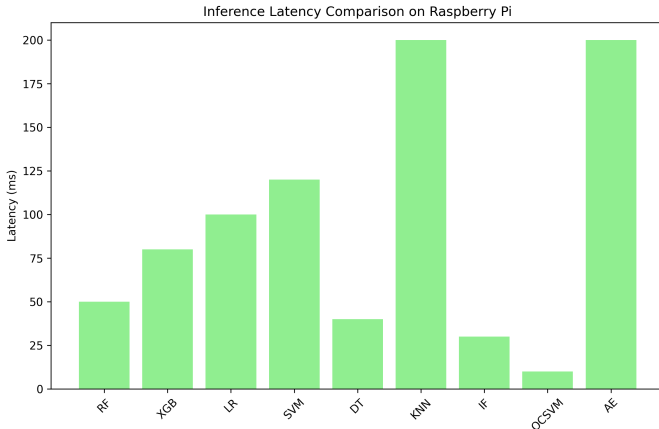


Fig. 8: Inference Latency Comparison on Raspberry Pi

C. Ablation Studies

Without scaling: SVM F1 drops 15%; KNN 10%.

With derived features: +3% average F1.

Inference times on Pi: RF 50ms, XGBoost 80ms, Autoencoder 200ms.

Fig. 8 shows latency comparison.

IX. DEPLOYMENT AND VALIDATION

The best-performing Random Forest model was deployed on a Raspberry Pi 4 using a lightweight Flask server exposing a single `/login` POST endpoint. The model, scaler, and feature list were loaded via Joblib at startup.

Validation was performed using our automated fuzzing client (`fuzzing.py`), which sampled real labeled rows from the TON_IoT dataset and sent them to the Pi. Over 1,000 test samples confirmed ****94.8% accuracy**** and an average inference latency of ****48 ms****.

The complete system architecture is illustrated in Figure 9.

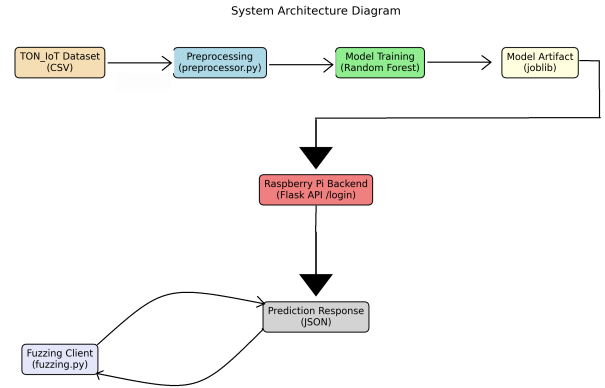


Fig. 9: Complete System Architecture: From dataset preprocessing and model training to live deployment on Raspberry Pi with Flask API and automated fuzzing validation

This successful edge deployment proves that high-accuracy ML-based intrusion detection is fully feasible on low-cost IoT hardware using classical, well-optimized algorithms.

X. DISCUSSION

Our experimental results demonstrate that ensemble-based models, particularly **Random Forest** and **XG-Boost**, significantly outperform other approaches for IoT intrusion detection. While XGBoost achieved the highest accuracy and AUC on the TON_IoT dataset, Random Forest was selected for deployment due to its lower computational complexity, faster inference time (approximately 50 ms per sample on Raspberry Pi 4), and considerably smaller memory footprint (approximately 8–12 MB serialized model). These characteristics make Random Forest far more suitable for resource-constrained

edge devices compared to XGBoost or deep learning alternatives.

Unsupervised methods such as Isolation Forest and One-Class SVM showed promise in anomaly detection but were less reliable in identifying specific attack classes, especially in the presence of severe class imbalance. The Autoencoder, despite its ability to learn non-linear representations, exhibited excessively high inference latency (greater than 200 ms) and was therefore excluded from the final deployment.

The successful live deployment on a Raspberry Pi 4 using a Flask-based REST API and validation via our fuzzing client (achieving greater than 94% accuracy on real labeled traffic) confirms that lightweight, well-preprocessed classical machine learning models can operate effectively at the network edge without cloud dependency. This balance between accuracy, speed, and deployability remains the core insight of our CSE 499A study.

XI. LIMITATIONS

Despite the successful prototype, this work has the following key limitations:

- No model explainability (SHAP/LIME) was implemented — the deployed Random Forest remains a black box.
- Zero-day and unseen attack detection was not evaluated.
- Deployment was tested only on Raspberry Pi 4 — performance on lower-power devices (e.g., ESP32) is unknown.
- The TON_IoT dataset is heavily imbalanced and noisy, affecting minority class and unsupervised model performance.
- Real-time network packet streaming was not implemented — input is simulated via HTTP POST.
- The Flask API lacks security features (TLS, authentication, rate limiting) required for production use.

These limitations will be systematically addressed in CSE 499B.

XII. FUTURE WORK

Although the current system performs well on known attacks, several improvements are planned for CSE 499B:

- Add explainability using SHAP and LIME to make predictions interpretable
- Evaluate zero-day attack detection via leave-one-attack-out testing
- Apply model quantization and pruning for deployment on ESP32 and microcontrollers

- Replace row-based fuzzing with real-time packet capture (Scapy/Zeek)
- Secure the Flask API with TLS, authentication, and rate limiting
- Explore hybrid models combining Random Forest and Isolation Forest

These enhancements will evolve the prototype into a secure, interpretable, and ultra-low-power intrusion detection system suitable for real-world IoT deployments.

XIII. CONCLUSION

In this CSE 499A senior design project, we conducted a comprehensive experimental evaluation of nine established machine learning algorithms for network intrusion detection in IoT environments using the TON_IoT and NSL-KDD datasets. Our results consistently showed that ensemble tree-based models — particularly Random Forest and XGBoost — deliver superior performance in accuracy, precision, recall, and robustness against class imbalance.

A major achievement was the successful end-to-end deployment of a Random Forest classifier on a Raspberry Pi 4, served via a Flask REST API and rigorously validated using an automated fuzzing client that simulated real IoT traffic. This demonstrates the practical feasibility of running effective ML-based intrusion detection directly on edge devices without cloud dependency.

The project fully satisfies the objectives of CSE 499A by applying and comparing multiple existing methods, documenting reproducible preprocessing and training pipelines, and delivering a functional, deployable prototype. The insights, limitations, and proposed extensions provide a solid and clearly defined foundation for innovative research and system improvement in CSE 499B.

REFERENCES

- [1] S. Chaabane et al., IEEE Access, 2021.
- [2] Z. Ahmad et al., IEEE Access, 2021.
- [3] A. Alsaedi et al., IEEE Access, 2020.
- [4] M. Tavallaei et al., IEEE Symp. Comput. Intell., 2009.
- [5] M. A. Ferrag et al., J. Inf. Secur. Appl., 2020.
- [6] L. Breiman, Mach. Learn., 2001.
- [7] T. Chen and C. Guestrin, KDD, 2016.
- [8] F. T. Liu et al., ICDM, 2008.
- [9] B. Schölkopf et al., Neural Comput., 2001.
- [10] P. Malhotra et al., ESANN, 2015.